

Project Report

*Tyler Cummings (14814578), Serena Chen (68696426), Advait Madhu (38690525),
Cooper Ross (29366861), Yash Kumar (66168568)*

1 Introduction

The Game of the Amazons is a two-player zero-sum game on a 10×10 board. Each side has four queens. On your turn you move one queen along an unobstructed line (like a chess queen), then shoot an arrow from that queen's new position along another unobstructed line. The arrow permanently blocks that square. If you can't move on your turn, you lose.

The main challenge for building an AI is the branching factor. In the early game there can be over 2,000 legal moves, so searching to terminal states is not feasible. This is similar to what was discussed in lecture about chess and Go. So we use depth-limited search with an evaluation function to estimate how good a position is instead of searching to the end.

The project has five classes. `COSC322Test` is the game client. `AmazonsBoardState` represents the board. `AmazonsMove` stores move data. `AlphaBetaSearch` does the search. `AmazonsEvaluator` evaluates positions.

2 Implementation

2.1 Game Client

`COSC322Test` extends the framework's `GamePlayer` and handles server communication. It listens for board state updates, game start messages, and opponent moves. When we receive an opponent's move we apply it to our internal board and check if it's our turn.

When it is our turn, we copy the board and pass it to `AlphaBetaSearch.chooseMove()`. We copy instead of passing the original because the search modifies the board internally (applying and undoing moves), and we don't want that interfering with the actual game state. The search returns the best move along with stats like depth reached and time taken, and we send the move to the server.

To figure out which side we're playing, we compare our username against the player names in the game-start message. The client also keeps the last 16 board hashes so the search can penalize repeated positions. We chose 16 because arrows make the board permanently different so repetition cycles in Amazons are short, and a small history is enough without wasting memory.

2.2 Board Representation

The board in `AmazonsBoardState` is an 11×11 integer array where indices 1 through 10 are the playable squares. Each cell is either empty (0), black queen (1), white queen (2), or arrow (3). We used 1-based indexing to match the server's coordinate system and avoid converting back and forth.

We keep two lists tracking where each side's queens are. Without these we'd have to scan all 100 squares every time we generate moves or evaluate, which adds up over thousands of calls. There's also an arrow counter we use as a game phase indicator, and a Zobrist hash updated incrementally.

For move generation, we iterate over the current player's queens and slide in all eight directions to find reachable destinations. For each destination we temporarily place the queen there and slide again for legal arrow shots. Each full move (from, to, arrow) becomes an **AmazonsMove**.

We also have **hasAnyMoves()**, which just checks if any queen has at least one adjacent empty square. This is much cheaper than full move generation and we use it for terminal detection in the search.

applyMove and **undoMove** update the board array, queen lists, arrow count, and hash. Undo reverses everything. We went with apply/undo instead of copying the board at every node because copying thousands of times per search would be too slow and create a lot of garbage for the JVM.

2.2.1 Zobrist Hashing

Every (row, column, piece type) combination has a random 64-bit number from a fixed seed. The hash is updated by XORing the relevant value when a piece goes on or off the board. There's a separate value for whose turn it is, since the same board with different sides to move is a different game state.

We use this hash both for looking up previously searched positions and for repetition detection.

2.3 Move Representation

AmazonsMove holds six integers for the queen source, destination, and arrow target. It converts to and from the server message format and implements **equals/hashCode** on all six fields, which the search needs for comparing moves in its ordering heuristics.

3 Game-Tree Search

The search is built around the minimax strategy from the lectures:

$$U(s) = \begin{cases} \text{Utility}(s), & \text{terminal} \\ \max_{a \in A(s)} U(\text{result}(s, a)), & \text{MAX node} \\ \min_{a \in A(s)} U(\text{result}(s, a)), & \text{MIN node} \end{cases}$$

Since we can't reach terminal states, we cut off at a certain depth and use an evaluation function instead. On top of basic alpha-beta we added several enhancements to make better use of the 30-second time limit.

3.1 Iterative Deepening

We use iterative deepening as described in the state space search lectures. The search goes depth 1, then depth 2, and so on until time runs out. If a deeper iteration gets interrupted we use the result from the last completed depth, so we always have a move ready.

The overhead of re-searching shallower depths is small compared to the deepest level, and each iteration provides useful information (best move ordering, cached results) that helps the next one go faster. So the earlier iterations actually pay for themselves.

3.2 Alpha-Beta Pruning

Alpha-beta keeps a window $[\alpha, \beta]$ at each node and cuts off branches that can't affect the result. From the lectures, α is a lower bound on MAX's guaranteed value and β is an upper bound:

```
1 int maxValue(State s, int alpha, int beta)
2     if (terminal) return U(s);
3     v = -INF;
4     for (Action a : A(s))
5         v = max(v, minValue(result(s, a), alpha, beta));
6         if (v >= beta) return v;
7         alpha = max(alpha, v);
8     return v;
```

We use a negamax formulation that combines `maxValue` and `minValue` into one function by negating and swapping the bounds. It's equivalent but easier to maintain.

With good ordering, alpha-beta reduces the effective branching factor from b to roughly $\sqrt{b}$, which as discussed in lecture means searching twice as deep. With random ordering it only reaches about $b^{3/4}$.

3.3 Additional Search Enhancements

Beyond what was covered in class, we implemented a few more techniques to get more out of the search.

Starting from iteration 2, we don't search the root with the full $(-\infty, +\infty)$ window. Instead we use a window of ± 250 around the previous iteration's score. A narrower window means more pruning. If the result falls outside we re-search with the full window, but the score usually stays stable so the narrow window pays off. We settled on 250 after trying different sizes as a balance between pruning benefit and re-search frequency.

We also use a technique where after searching the first child at each node (the one we expect is best), we search the remaining children with a zero-width window $[\alpha, \alpha + 1]$ to quickly verify they're worse. If one turns out to be better we re-search it with the full window. This doesn't happen often when the ordering is decent, so it saves time.

We also cache search results in a hash map keyed by Zobrist hash. Each entry stores the score, depth, bound type (exact/lower/upper), and best move. When we encounter a position we've already searched deep enough, we can return the stored result or use it to narrow the window. Even when we can't skip the node, the stored best move helps with ordering. The cache holds 200,000 entries with LRU eviction and persists across iterations but gets cleared between games.

3.4 Move Ordering

As covered in class, alpha-beta's effectiveness depends heavily on evaluation order. We use two phases.

First, every move gets a fast score without being applied to the board. The cached best move for this position gets the biggest bonus since it was best last time. The best move from the previous iteration also gets a large bonus. We keep two "killer move" slots per depth level for moves that

caused beta cutoffs at the same depth in other branches, since if a move was strong enough to cause a cutoff in a sibling branch it might be strong here too. There's also a history heuristic tracking which moves have caused cutoffs throughout the search, weighted by $64 \times \text{depth}^2$ because cutoffs at higher depths save more work. A small center-bias term is added since central positions tend to give better mobility in Amazons.

After sorting by this cheap score, the top candidates get a second pass where we apply each move and compute a more informed score. This checks if the opponent has any replies (instant win), counts the queen's mobility at its new square, looks at active queen counts, and penalizes moves leading to repeated positions. We only do this for the top candidates since applying each move is expensive.

We cap moves at each node to 32–40 at interior nodes and 160 at the root. We're throwing away most legal moves, which risks missing something, but without this the branching factor is too high to search more than a couple levels deep. A deeper search with fewer moves tends to play better than a shallow search with all moves.

4 Evaluation Function

The evaluation function approximates the minimax value at leaf nodes. As mentioned in the lectures, the min-distance function via BFS from each queen is a useful feature for Amazons. We use this as the main component with several additional features.

4.1 Territory via BFS Distances

For each side we run a multi-source BFS from all queens simultaneously, computing how many queen moves it takes to reach every empty square. This gives two distance grids.

Each empty square gets classified by comparing the distances. If only one side can reach it, it belongs to them. If both can but one is closer, it goes to the closer side. If they're tied, we look at which side has more queens nearby to break the tie, since having a queen close means you're more likely to actually claim that square.

Territory is the most heavily weighted feature and its weight increases as the game progresses. This makes sense because early on the board is open and boundaries are fluid, but once arrows partition things the territories become permanent.

4.2 Separated Positions

If no square is reachable by both sides, the queens are completely walled off from each other and each side is just filling in their own area.

We handle this as a special case because the evaluation becomes much simpler and more accurate. We count how many empty squares each side can reach and multiply the difference by 500. Each reachable square is one more move that player can make before running out of room, so the side with more space wins.

4.3 Other Features

Raw mobility counts the total squares each queen can reach in one move. Local mobility is the same but capped at 12 per queen, because without the cap one queen in a wide open area could dominate the score and hide the fact that the others are stuck.

We count “active” queens (those with at least one destination). Trapped queens (zero destinations) get a heavy penalty (weight 120) and nearly-trapped queens (four or fewer) get a smaller one (weight 45). We also check how many directions around each queen are open and how far they go, since a queen with only one or two ways out is vulnerable even if not technically trapped.

A frontier feature gives extra weight to contested squares near arrows or board edges, since these tend to be chokepoints that determine where territorial boundaries end up. A region control feature uses flood fill to find connected empty regions reachable by only one side, giving a bonus proportional to size since a larger exclusive region means more moves for that player.

4.4 Phase-Dependent Weights

The final score is a weighted sum of these features using the arrow count as a phase indicator. Territory weight goes from 60 to 180 over the game. Mobility goes from 12 to 4. Mobility matters early when queens move freely, but territory takes over once arrows split the board.

All scratch arrays (BFS queues, distance grids, region markers) are stored in a `ThreadLocal` and reused because the evaluator runs tens of thousands of times per move. Allocating fresh arrays each time would put too much pressure on the garbage collector.

5 Time Management

The server gives 30 seconds per move. We use a 27-second soft limit for margin. The budget per move depends on the number of legal moves and the game phase. Fewer legal moves or more arrows on the board means less time needed. There’s a 4-second minimum so the search always has something to work with.

If the best move changes between iterations or the score shifts significantly, we extend the deadline so the search can stabilize. If results have been consistent for a couple depths and we’ve used most of our time, we stop early instead of starting another iteration that probably won’t change anything. We also stop early if the score is clearly winning or near the win/loss value.

6 Limitations and Possible Improvements

There is no special handling for the search horizon. When we hit the depth limit we just evaluate, even if a queen is about to get trapped on the next move. Adding a one-level extension when a queen is nearly trapped could help.

Move generation is wasteful. We build the complete list at every node and throw most of it away during ordering. Generating arrow shots lazily, only for queen destinations that scored well, would reduce unnecessary work.

We didn’t implement late-move reductions (searching the tail of the ordered move list at reduced depth). Since our ordering already puts the good moves first, the later moves probably don’t need

full-depth search.

The search is single-threaded. The code structure would support parallelization but we didn't have time.

It might also be worth trying to learn the evaluation weights through self-play instead of hand-tuning, similar to the reinforcement learning approaches from class.

7 Conclusion

Our player uses depth-limited minimax with alpha-beta pruning, iterative deepening, cached search results, and several move ordering heuristics. The evaluation function is based on BFS territory analysis with mobility, queen safety, and region control features weighted by game phase. We handle the large branching factor by aggressively pruning the move list while using heuristics to keep the most promising moves. Time management adapts to position complexity and stops early when results are stable.